National University of Lesotho

Department of Mathematics and Computer Science

CS4433 Software Engineering — Final Project

A Moderated Learning Management System

A C++17 / SQLite console application taken through the full software engineering life cycle

Author: Mweli Pama (201404452)

Repository: https://github.com/mwelipama/cs4433-moderated-lms

Release v1.1 — repository main branch

Date: November 2026

Abstract

This report documents the development of a Moderated Learning Management System (LMS) for a secondary school, carried out as an individual project through a Waterfall life cycle with V-Model verification. The system serves four roles — Administrator, Lecturer, Head of Department and Student — and enforces a moderated assessment workflow in which no mark is visible to a student until a Head of Department has released it. It is implemented in C++17 on an embedded SQLite database, using a layered architecture and the Strategy, Repository/DAO and Factory patterns. Passwords are stored as Argon2id hashes, all mark changes are written to an append-only audit trail, and marking, moderation and release each execute as a single atomic transaction. The business-logic layer is covered by an automated test suite (38 cases, 112 assertions) with 80.9% measured line coverage, continuous integration passes on Ubuntu and Windows, and the application has been exercised end-to-end against a real database including persistence across restarts. Change request CR1 (class-wide deadline extensions) is implemented as release v1.1. The report summarises each phase, states the evidence honestly, and records the one deliberate deviation from the specification.

Contents

1. Introduction · 2. Project Management Plan · 3. Requirements Engineering · 4. Software Design · 5. Implementation · 6. Verification and Validation · 7. Configuration Management and Quality Assurance · 8. Deployment · 9. Maintenance (CR1) · 10. Professional, Legal and Ethical Issues · 11. Conclusion · 12. References · Appendices (traceability, test results, user manual, risk register)

1. Introduction

A secondary school requires software to manage courses, assignments and assessment for roughly 800 users on old computers with unreliable internet. The central rule is moderation: a student must never see a mark before a Head of Department (HoD) has checked and released it. The system must also accept and penalise late work fairly, allow resubmission before a deadline, support individual and class extensions, and keep a complete record of who changed each mark, when and why.

The project was taken through the full life cycle using the Waterfall model with V-Model verification. The deliverable is a maintainable, secure, object-oriented C++17 console application with persistent SQLite storage. This report presents each phase in condensed form and is accompanied by the Git

repository, which holds the requirements and design documents, the source code, the test suite, the UML diagrams and the deployment and quality-assurance evidence.

2. Project Management Plan

A feasibility study found the project technically feasible (a standard C++/SQLite toolchain), operationally feasible (a console interface suits non-expert staff and old hardware), and economically feasible (only open-source tools). The work was decomposed into a Work Breakdown Structure and scheduled across seven weeks with three supervisor phase-gates; design was not started until requirements were accepted, nor implementation until design was accepted. A risk register tracked technical, project and people risks — the most significant being scope creep, requirement conflicts and the security of assessment data — each with likelihood, impact, mitigation and owner, and was revisited at every gate. The updated register is in the appendix.

3. Requirements Engineering

Five stakeholders were analysed (Principal, Lecturer, HoD, Student representative, IT officer) and seven conflicts in their statements were resolved explicitly. The three decisive resolutions were: (a) the late-penalty rule — 10% of the maximum mark per started day late, to a maximum of three days, after which work scores zero and is not accepted (a 72-hour cutoff); (b) marks are released only by the HoD, so a submission's state is the single authority for visibility; and (c) a student may resubmit freely up to the deadline, but after it no existing submission (on-time or late) may be replaced, and a first late submission is accepted only when none exists.

The Software Requirements Specification follows the ISO/IEC/IEEE 29148 structure and defines 21 functional requirements and 10 non-functional requirements (plus FR-22 for CR1), each uniquely identified, prioritised with MoSCoW and given a verifiable acceptance criterion. Non-functional requirements are measurable — for example, login completing within two seconds with 800 users stored on the reference machine. A use-case model covers all four actors with include, extend and generalisation relationships, and three use cases are fully dressed: Submit Assignment (with on-time, late and refused-replacement flows), Mark Submission and Moderate and Release.

4. Software Design

The chosen architecture is a four-layer design — Presentation, Services, Data Access (DAO) and Domain, over an Infrastructure layer. A Model–View–Controller style was considered and rejected: the system has no graphical views and its complexity lies in business rules and persistence, which a service/DAO separation expresses more directly. The design applies three patterns, each justified: Strategy isolates the late-penalty rule behind a PenaltyPolicy interface; Repository/DAO confines all SQL and the parameterised-query guarantee to one layer; and Factory (UserFactory) centralises the role→subclass mapping. The design follows the SOLID principles — services depend on repository and policy interfaces (Dependency Inversion), and a new penalty rule is a new class rather than a change to marking (Open/Closed).

An abstract User class with Administrator, Lecturer, HeadOfDepartment and Student subclasses provides the required inheritance and polymorphism. Behaviour is modelled by a state machine for

the submission life cycle (Submitted → Late → Marked → Moderated → Released), a swimlane activity diagram, and sequence diagrams for submission and for moderation/release. The database is an ERD normalised to Third Normal Form with a data dictionary and an SQL schema whose tables map directly to the domain classes; the audit trail is append-only, enforced by triggers, and a mark's visibility is derived from its submission state.

5. Implementation

The application is implemented in C++17 and built with CMake. The source is organised by layer: infra/ (SQLite connection and prepared-statement wrappers, schema initialisation and migration, password hashing, datetime, secure-random), domain/ (the User hierarchy, entities, enumerations, the penalty Strategy and the Factory), dao/ (repository interfaces and SQLite implementations), services/ (authentication, administration, submission, marking, moderation and audit), ui/ (the role-based console) and app/ (the entry point). The code base is about 3,900 lines excluding vendored dependencies, with an average cyclomatic complexity of 2.3.

Key design decisions. (1) SQLite and the Argon2id hasher are vendored and compiled from source, so the application has no external dependencies and a school technician can build it with only CMake and a C++17 compiler. (2) All data access uses parameterised prepared statements confined to the DAO layer. (3) Marking, moderation and release each run inside a single database transaction (the commit is inside the try block so a failed commit also rolls back), so a mark and its audit event are always written together and a release never partially commits. (4) The submission state column is the sole release authority.

Security. Passwords are hashed with Argon2id and stored as PHC strings that embed the algorithm, parameters and a per-user salt; the salt is drawn from the operating-system CSPRNG (BCryptGenRandom on Windows, /dev/urandom on POSIX). Role and ownership checks are enforced in the service layer — a lecturer only on assigned courses, a HoD only within their department, a student only on enrolled courses, including the read-only listing paths. Invalid input (out-of-range marks, empty feedback, impossible calendar dates) is rejected through a typed exception hierarchy so the program never crashes, and data persists in the SQLite file between runs.

Deviation from specification. NFR-03 names libsodium for Argon2id. The implementation instead vendors the Argon2 reference implementation behind an IPasswordHasher interface — a conscious deviation for build portability on the school computer (system libsodium is a non-trivial dependency to install on Windows). It produces and verifies the same Argon2id hashes with the same parameters and, being behind the interface, can be switched to libsodium without touching the services. It is recorded as a deviation, not as full NFR-03 compliance.

6. Verification and Validation

The test strategy maps to the V-Model: unit tests verify the penalty policy, datetime and lateness arithmetic, the submission rules and the audit model; integration tests exercise the services against a real SQLite database; system and acceptance checks derive from the use-case flows including exceptions. Equivalence partitioning and boundary-value analysis cover the time-sensitive rules (at

the deadline, one second late, exactly 24 and 72 hours, beyond the cutoff) and the penalty at zero, one and three days.

Results. The automated suite (doctest) runs 38 test cases and 112 assertions, all passing. Measured line coverage of the business-logic layer (services and domain) is 80.9%, exceeding the 70% target; overall src/ line coverage is 58.3%, the difference being the interactive console, which is validated manually. Continuous integration builds and runs the whole suite on Ubuntu and Windows on every push, and both pass. The application was exercised end-to-end through all four roles against a real database: a student saw the moderated mark (85, not the original 80) and only after release; persistence was confirmed by reopening the same database file in a fresh process. A defect log is in the appendix.

7. Configuration Management and Quality Assurance

The project is version-controlled in a public Git repository with descriptive, gated commits. Development uses a feature-branch and pull-request workflow: CR1 was implemented on a branch and merged through pull request #1 with continuous integration passing. Continuous integration (GitHub Actions) builds and tests the project on Ubuntu and Windows on every push and is green. Quality assurance included an independent review that raised three findings — a libsodium deviation (disclosed), an access-control gap in two listing paths, and the use of std::random_device for salts — the latter two fixed with regression tests. cppcheck static analysis was run and addressed to zero findings (one justified suppression); code metrics show about 3,900 lines, average cyclomatic complexity 2.3 and no function above the complexity-15 threshold. A documented coding standard and a completed self-review checklist are in docs/quality/. Releases are tagged v1.0 and v1.1.

8. Deployment

The system deploys as a single self-contained executable plus one SQLite database file, on one school computer, entirely offline. A technician builds it with two CMake commands and runs ./build/lms; on first run the program creates the database, applies its schema and prompts for the initial administrator account. There is no server, service or network dependency, and the database file is the single item to back up. Because all dependencies are vendored, the same source builds on Windows, Linux and macOS. A UML deployment diagram, installation guide, per-role user manual and release notes (v1.0 and v1.1) are in docs/deployment/.

9. Maintenance (Change Request CR1)

CR1 asks that a lecturer be able to extend a deadline for a whole class, not only an individual student, and that extensions appear on the student's assignment list. Classification: a perfective (enhancement) change. It is implemented as release v1.1: a class_extensions table (one deadline per assignment) added by a safe idempotent migration so existing databases upgrade automatically; a lecturer-authorised extendClass operation; and the effective deadline extended to the latest of the base, class and individual deadlines — applied identically to submission acceptance and to the late penalty through one shared function, and shown on the student assignment list. An individual extension still applies, and where both exist the student keeps whichever deadline is later. Six

regression tests cover the class extension, authorisation, precedence, penalty flow-through, a control case and persistence; the change was merged through pull request #1 with CI passing. The SRS (FR-22), use-case model (UC-20), database design and traceability were updated accordingly.

10. Professional, Legal and Ethical Issues

The system holds learners' personal information and assessment records. Personal data is minimised to what the workflow needs, passwords are stored only as Argon2id hashes, and access is restricted by role and ownership. The integrity of assessment records is protected by the append-only audit trail, which makes every mark change attributable and tamper-evident, and by the rule that only a HoD can release a mark. Accessibility is served, within scope, by a simple text interface that runs on old hardware; a future graphical interface should meet recognised accessibility guidelines. Fairness of the late-penalty rule is addressed by making it transparent, applied consistently by the system rather than at a marker's discretion, and tempered by documented individual and class extensions. Measured against Lesotho's Data Protection Act, 2012 (Act No. 5 of 2012), the design supports lawful-purpose and minimised data handling, confidentiality and integrity (hashing, access control, the append-only audit) and accountability; the controls implemented here are technical, while organisational duties such as retention schedules, backup custody and breach response remain the school's responsibility. No claim of legal or compliance certification is made.

11. Conclusion

The project delivers a working, moderated LMS that meets the Must requirements: role-based access for four user types, the agreed late-penalty, extension and resubmission rules, HoD moderation and controlled release, persistent SQLite storage through a data-access layer, Argon2id password hashing, an append-only mark audit trail, and exception-based robustness, built with CMake and covered by an automated test suite with 80.9% business-logic coverage. CR1 is implemented as release v1.1, and continuous integration, static analysis and the deployment artifacts are in place. The one deliberate deviation is the vendored Argon2id implementation in place of libsodium (section 5); that, and the usual limits of a single-developer timeline, are the honest remaining caveats. The core workflow is complete, tested and demonstrated.

12. References

1. ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
2. A. Biryukov, D. Dinu and D. Khovratovich, Argon2 (RFC 9106), IETF, 2021.
3. OWASP Foundation, Password Storage Cheat Sheet, 2024.
4. E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns, Addison-Wesley, 1994.
5. R. C. Martin, Agile Software Development: Principles, Patterns, and Practices (SOLID), Prentice Hall, 2003.
6. Kingdom of Lesotho, Data Protection Act, 2012 (Act No. 5 of 2012).
7. SQLite Documentation, https://www.sqlite.org/docs.html.
8. I. Sommerville, Software Engineering, 10th ed., Pearson, 2015.

Appendices

A. Requirements Traceability (sample)

Requirement	Use case	Design element	Source file	Test
FR-12 Resubmission/late	UC-10 Submit	SubmissionService	services/ SubmissionService.cpp	test_submission_rules
FR-13 Late penalty	UC-08 Mark	Strategy: StandardPenaltyPolicy	domain/ StandardPenaltyPolicy.cpp	test_penalty_and_time
FR-16 Release authority	UC-12/13	ModerationService, state col	services/ ModerationService.cpp	test_moderation_and_release
FR-17 Mark audit	UC-08/12	AuditService, audit_events	services/ AuditService.cpp	test_audit_integrity
FR-22 Class extension (CR1)	UC-20	AssignmentService:: extendClass; effectiveDeadline()	services/ AssignmentService.cpp	test_class_extension
NFR-03 Password hashing	—	Argon2idPasswordHasher	infra/ Argon2idPasswordHasher.cpp	test_auth
NFR-04 Parameterised SQL	—	DAO (prepared statements)	dao/ SqliteRepositories.cpp	(all integration)

B. Test Results and Defect Log

Automated suite: 38 cases / 112 assertions, all passing; 80.9% business-logic line coverage; CI green on Ubuntu and Windows; clean build on GCC 11.4 and 13.3; end-to-end four-role walk and restart-persistence passed.

#	Defect / finding	Severity	Cause	Fix	Status
1	Assignment listings bypassed authorisation	Medium	Console read called repository directly	Enrolment/ department checks in the service layer	Fixed + test
2	Salt from std::random_device	Medium	Non-cryptographic source possible on some platforms	OS CSPRNG (BCryptGenRandom / /dev/urandom)	Fixed + test
3	Argon2id vendored, not libsodium (NFR-03)	Low	Portability choice for the school computer	Isolated behind IPasswordHasher; switchable	Disclosed
4	Impossible calendar dates accepted	Low	Range check lacked day-of-month validation	Validate day against month and leap year	Fixed + test
5	Terminal echo not restored on a throw	Low	Restore only on the normal path	Restore in a catch handler	Fixed
6	A failed COMMIT could skip rollback	Low	COMMIT was outside the try block	Moved COMMIT inside the transaction try	Fixed

C. User Manual (by role)

- Administrator: create departments, then users (HoD requires a department), courses, lecturer assignments and enrolments; list and activate/deactivate accounts.
- Lecturer: list assigned courses; create assignments; grant an individual extension; extend the deadline for a whole class (CR1); view the queue; mark submissions (feedback required; a reason is required to change a mark).
- Head of Department: list department courses; view the moderation queue; adjust a mark (with a reason) or accept it unchanged; release an assignment's moderated marks.
- Student: list enrolled courses and assignments (effective deadline shown); submit or resubmit before the deadline; view submission status; view the released result and feedback.

D. Risk Register (updated)

Risk	L	I	Mitigation	Status
Requirement conflicts cause design churn	M	H	Resolve all conflicts at Gate 1; traceability	Closed
Scope creep beyond the case study	M	M	MoSCoW; gate reviews; fixed scope	Controlled
Security of assessment data	M	H	Argon2id, service-layer authz, append-only audit	Mitigated
Dependency/build portability on school PC	M	M	Vendor SQLite + Argon2; CMake only; CI on Windows	Closed
Single-developer time pressure	H	M	Gate-by-gate delivery; prioritise Must reqs	Active